关于 LofiSu：一个靠包装形象实现上位的欺诈者

Codex Yonagi04 2026年5月23日 11:40 广东

注：以下内容是作者认识的一位同学在使用 Codex 挖掘 Github 用户 Lofisu 的历史 Commit、PR、Issues记录后自动生成的一篇文章，为保护该同学身份，作者代为发布，因此本文不代表本作者观点。如有疑问请咨询OpenAI的奥特曼。

如果你不知道Lofisu是谁，以及不知道关于此人的节奏，请看请看【喜提xx在coding拉黑 – 小概率事件 | 小红书 – 你的生活兴趣社区】 😆 DzeeYPQzbjoaBuO 😆 https://www.xiaohongshu.com/discovery/item/6a102118000000000360338b7?source=webshare&xhsshare=pc_web&xsec_token=CBeDokCm1qFist5kVvMvB3Z-505NgL9ya3aBE2m726W-k=&xsec_source=pc_share

> 前言： 她不是空壳，也不是纯骗子；她有真实产出，尤其集中在 **站点前端、文档、交互和页面型实现** 。
> 但她最强的能力，不是某个 **单点技术标签** ，而是把 **有限但真实的经历** ，排布成一个 **明显更高位的身份叙事** 。
> 把 PR、issue、README、个人 GitHub repo 结构和 connection 方式放在一起看，会发现她很会占位、抬位、进入高势能项目，再把这些转成外部形象。
> 所以我真正反感的，不是她“会包装”，而是她把 **能力、关系、头衔都工具化成上升材料** 。

如果我完全不认识她，只看公开材料，我对她的第一印象不会是“骗子”，而会是另一种更典型、也更复杂的人：

她有真实材料，有执行力，有一些能拿得出手的公开产出，也非常懂得怎么把这些产出重新排列，最后制造出一个明显高于事实边界的整体印象。

换句话说，她的问题恰恰不在于“完全没有东西”，而在于她不是空的，所以她的包装不是凭空捏造，而是更麻烦的一种形式：把真实但边界明确的经历，讲成更高位、更完整、更有力量的身份叙事。

我对她的整体判断，也正是从这里开始的。

这篇文章想把之前那几篇拆开的内容重新合成一条完整叙事线：

1. 只看公开事实，我对她的第一印象是什么。
2. 她到底做过哪些真实可核验的事。
3. 她 README 里哪些话成立，哪些话成立但容易误导，哪些话我不能替她确认。
4. 她是怎么一步步把自己从“前端开发”讲成“AI Full-Stack / Apache Committer / DeerFlow Maintainer / AI Infra”的。
5. 最后，为什么即便承认她有真实产出，我依然对这种人保留很强的反感。

一、先只看事实：我对她的第一印象是什么

如果只看公开记录，不看任何私下故事，我会先得出三个很明确的印象。

1. 她不是空壳

这一点必须先说清楚。

她在公开仓库里确实有真实记录，而且不是完全靠嘴：

- `apache/fory-site` ：公开可见 `12` 次 contribution、 `18` 个 PR
- `bytedance/deer-flow` ：公开可见 `100` 次 contribution、 `5` 条作者 PR
- `Tencent/omi` ：公开可见 `2` 次 contribution、 `2` 条 PR
- `alibaba/spring-ai-alibaba` ：公开可见 `2` 条 PR

所以如果有人想把她概括成“纯空壳”“全靠吹”，我觉得这是不准确的。她不是没有东西。

2. 她最扎实的公开工作重心，其实相当一致

如果把这些仓库放在一起看，她最稳定、最容易核实的工作类型，高度集中在：

- 站点前端
- 文档与国际化
- 页面组件与响应式适配
- Citation / thread / settings / navigation 这类产品交互层功能
- 页面型、工作流型、可视化型实现
- 中间件 bugfix 和周边 API

这组重心本身没有问题，甚至可以说是很扎实的一类产出。但它和她后来对外呈现出来的高位身份之间，存在明显的叙事跃迁。

3. 她非常懂怎么讲故事

如果一个人的 profile 仓库总提交大约 `282` ，其中 `README.md` 一项就占 `150` 次，且仅 `2026-03-09` 一天就改了 `13` 次 README，这基本已经不是“顺手写个主页”了。

这说明她在做一件非常明确的事：

> 她在持续经营一个面向招聘市场和流量市场的个人品牌页。

这就是我只看事实得出的第一印象：
不是“假”，而是“会”；不是“空”，而是“很懂怎么把有限但真实的东西排布成一个更高位的自己”。

二、公开事实核查：她到底做过哪些真实可核验的事

1. Apache：身份是真的，但公开最扎实的证据主要在官网站点侧

先说结论： `Apache Committer` 这个身份本身是真的。

ASF 官方 committer 名单里确实有 `lofisu` ，而且挂在 `fory` / `incubator` 名下。所以这件事不能直接说她是假的。

但如果继续往下看公开贡献，问题就出现了。

她最扎实的 Apache 公开记录，主要不在 `apache/fory` 主仓库，而在 `apache/fory-site` 官网仓库。

`apache/fory-site` 当前公开能看到的是：

- `12` 次 contribution
- `18` 个 PR
- 活跃高峰大致集中在 `2024 年 8 月到 11 月`
- `2025 年 4 月` 和 `2025 年 6 月` 还能看到零星继续

更关键的是，PR 内容非常统一，集中在官网和站点侧，而不是主仓库核心逻辑。公开能点开的 PR 包括：

- `#141` 国际化配置和简体中文版本
- `#147` 官网动画效果与 FAQ 文档
- `#152` 新增 user page
- `#177` 首页语言卡片和代码展示模块
- `#181` JavaScript icon 404 与复制按钮
- `#193` 语言 quick start 路径
- `#198` 首页语言卡片路由跳转
- `#202` 移动端 CSS 适配
- `#213` 组件结构重构、Tailwind 响应式、性能优化
- `#216` 文档版本化支持，最终 `closed`
- `#264` 首页 Scala/Kotlin 代码示例

也就是说，如果你看她 Apache 这部分最强的公开记录，能看到的是：

- 站点前端
- 国际化
- 组件
- 文档
- 首页展示
- 响应式和视觉交互

反过来看 `apache/fory` 主仓库，公开信号就弱很多：

- 默认分支按作者筛 commit： `0`
- 公开 PR： `1`
- 那 `1` 条还是 `#1867` ，标题是 `docs: Update Rust README.md`
- 最终状态： `closed` ，未合并
- 改动文件：只有 `rust/README.md`

所以更准确的说法不是“她只是改过官网，不算 Apache Committer”，而是：

> 她的 Apache 身份是真的，但她公开最强的贡献证据，主要支撑的是 Apache 官网/站点建设，而不是主仓库核心实现。

这就是第一处叙事落差。
身份是真的，但外界最容易脑补出来的那种“深度参与核心引擎”的印象，并不由当前公开记录直接支持。

2. DeerFlow：贡献很多，但重心明显偏前端、交互、状态和周边 API

`DeerFlow` 是她公开材料里最容易让人觉得“这个人很猛”的部分。

因为数字本身确实不低：

- contributors 里公开显示 `100` 次 contribution
- 公开作者 PR 目前能稳定查到 `5` 条

具体是：

- `#1063` `feat: memory management API with injection toggle and fact deletion`
- `#1064` 同题，当前仍是 `open`
- `#1077` `fix(frontend): fix new-chat navigation stale state issue`
- `#1081` `fix(middleware): 修复 runtime.context 为 None 的崩溃问题，并解除标题生成对对话的阻塞`
- `#1088` `fix(frontend): fix new-thread history 404/422 errors`

如果继续看 PR 正文，会发现她做的也不是没有技术含量的小修。比如：

#1063 / #1064

公开正文里明确写了：

- 新增 `PATCH /api/memory/config`
- 新增 `DELETE /api/memory/facts/{id}`
- 修复 `injection_enabled` 的跨进程竞态问题
- 前端记忆设置页新增“在对话中注入记忆”开关
- Fact 从 Markdown 表格改成交互式列表
- 配套改了 memory 相关前后端代码和测试

这说明她不是只在 README 或页面文案层面打转，这条线里确实碰到了接口、状态持久化、前后端联动和测试。

#1077

这条是已合并 PR。正文写得很具体：

- 把 `history.replaceState` 改成 `router.replace()`
- 修复 Next.js 内部路由状态不同步
- 调整 `ChatLayout` remount 逻辑

这是一个很明确的前端导航状态 bugfix。

#1081

这条虽然没合并，但正文也说明了她碰过 middleware 层问题：

- 修复 `runtime.context` 为空导致的崩溃
- 把标题生成从同步阻塞改成异步后台任务，避免对话被卡 `5-10 秒`

#1088

这条也是很典型的前端线程状态问题：

- 修复新线程首次发送消息时触发异常 `history` 请求
- 对应 `404 / 422`
- 关联到 `fetchStateHistory` 、Agent chat 页的 `history.replaceState` 和线程状态残留

但关键不在于“她有没有做事”，而在于：
把这些 PR 标题、正文和改动文件放在一起看，方向其实非常统一。

她在 DeerFlow 的公开最强重心，明显落在：

- `frontend`
- `citations`
- `thread state`
- `memory settings`
- `navigation`
- `UI / interaction`
- `middleware bugfix`
- 周边 API

这并不等于这些工作不重要。问题在于，这和外界一看到 `Maintainer of ByteDance/DeerFlow` 、 `AI Infra` 时，自动脑补出来的那种“核心底层维护者”形象，并不是一回事。

更关键的是，DeerFlow README 当前公开列出的 `Key Contributors / core authors` 是：

- `Daniel Walnut`
- `Henry Li`

公开列名里，我没有看到她被写进 core authors。

所以这件事更准确的结论是：

- 她在 DeerFlow 绝对不是零贡献，也不是轻飘飘蹭热度。
- 她做过的东西里有一部分确实是实打实的前后端联动和问题修复。
- 但从公开证据看，她的可见重心仍然明显偏前端、交互、状态管理和周边 API。
- `Maintainer` 这个词，按我现在能独立核实到的公开页面，证据仍然不够。

也就是说，这一项不是“完全虚构”，而是：

> 贡献真实，而且不轻，但头衔强度仍然写得偏高。

3. Tencent Omi：贡献存在，但体量不大

`Tencent/omi` 这部分也值得单独说，因为它很典型。

公开能看到的是：

- `2` 次 contribution
- `2` 条 PR： `#913` 、 `#914`
- 其中 `#914` 于 `2024-07-20` 合并

`#914` 的 PR 正文写得很清楚：完成的是一个 `TDesign` 风格企业官网模板 issue。
改动文件也只有这几个核心文件：

- `packages/omi-templates/src/pages/company.tsx`
- `packages/omi-templates/src/routes.tsx`
- `packages/omi-templates/src/store.ts`

这说明 `Open Source Contributor – Tencent Omi` 这半句是成立的。
但它支撑的是一条体量不大的模板页面 PR 线，而不是一个分量很重、持续时间很长的项目角色。

4. Spring AI Alibaba：有记录，但仍然指向页面和交互层

这部分比我一开始预想的稍微多一点。

公开能看到 `2` 条 PR：

- `#189` `Doc : Added contribution guidelines document`
  - 已合并
  - 改了 `CONTRIBUTING.md` 、 `README.md` 、 `README-zh.md`
- `#208` `[Feat] Complete Low-Code AI Workflow Page Implementation`
  - 低代码 AI workflow 页面
  - 背景画布和 mini-map
  - 节点 CRUD
  - NodeEditorDrawer
  - 连接线和交互逻辑
  - 后续保存/加载配置预留

  - 已关闭，未合并
  - 正文非常长，内容包括：
  - 改动文件集中在 `graph-ui/src/pages/Graph/Design/*`

这条信息很关键，因为它再次把她的公开工作重心拉回到了一个非常稳定的方向：

- 页面
- 交互
- 可视化
- 工作流编辑器
- 产品侧前端

而不是大家最容易脑补出来的“高位基础设施角色”。

5. Higress 和 OpenYurt：主仓库层面当前公开证据偏空

这两项恰好能说明她 README 里最典型的一种包装方法。

至少在当前主仓库公开作者记录里：

- `alibaba/higress`
  - 公开 PR： `0`
  - 公开作者 commit： `0`
- `openyurtio/openyurt`
  - 公开 PR： `0`
  - 公开作者 commit： `0`

这并不绝对排除她在别的关联仓库、官网仓库、镜像仓库或者其他身份邮箱下做过事。
但如果写严谨版，就不能把它们直接写成“我已经在主仓库层面核验过的贡献”。

这也是她公开叙事中一个很典型的技巧：

> 把强弱差异很大的几段经历并排摆，最后给外部读者制造出一种“每一项都很扎实”的整体观感。

6. Issue 行为：她不只是提 PR，也会先用 issue 占位、试探需求、建立角色感

这部分是我后面继续用 `gh` 补查时，才更明确确认下来的。

她在这些相关仓库里，不是只提交 PR，也会主动发不少 issue。
而且这些 issue 的风格非常统一。

如果粗分，大概有两类。

第一类，是相对正常的 bug 反馈或使用问题：

- `spring-ai-alibaba#619` ： `Service接口缺失StudioUi无法启动`
- `openyurt.io#443` ：本地 `npm run serve` 后改代码浏览器不刷新
- `higress-group.github.io#306` ：按钮文字在不同设备上的换行问题

这类 issue 说明她并不是完全不碰真实问题。

但更值得注意的是第二类，也是她更高频的一类：

- **“这个需求要不要做，我可以做”**
- **“官网要不要 team page，我可以写”**
- **“要不要 AOS 动画 / 国际化 / Twind 响应式 / Graph UI 方案”**

典型例子包括：

- `apache/fory-site#139` ：**希望官网支持中英文切换，并明确写“如果社区需要的话我可以做这个 issue”**
- `apache/fory-site#149` ：**提 team page，并写** `I will submit PR within two days`
- `openyurt.io#446` ：**问官网要不要 team page，并写** `I can write one`
- `openyurt.io#441` ：**问官网要不要 AOS 动画，并写** `I can try to pull request`
- `spring-ai-alibaba#162` ：**直接问有没有前端开发需求、前端架构是什么、能不能给前端 issue 打标签，明确争取进入项目**
- `spring-ai-alibaba#170` ：**主动给 Graph UI 提技术选型建议**
- `higress-group.github.io#294` ：**问官网要不要 team page**
- `higress-group.github.io#287` ：**问官网要不要 AOS 动画**
- `apache/fory#1793` ：**直接开了一个** `Fury FAQ` **issue，后来还被维护者反问** `Do we really need this issue?`

把这些 issue 放在一起看，会发现她有一个非常稳定的动作模式：

- **先进入维护者视野**
- **先把“我有想法”“我能做”“我正在参与”公开摆出来**
- **再把后续 PR、页面实现或个人履历，接到同一条线里**

这套动作本身并不违规。
在开源社区里，先提 issue 再做 PR，本来就是正常操作。

但问题在于，当它和她后来的 README 叙事、头衔排布、connection 建立方式放在一起看时，它呈现出来的就不是单纯的“正常参与”，而更像一种很成熟的角色经营：

> 她不是在等别人给她一个位置，而是会主动寻找一个低风险、高可见度、又正好适合自己前端/官网型能力发挥的位置，先用 issue 占位，再用 PR 把这个位置坐实。

这会进一步解释我为什么一直觉得，她最强的能力之一，不只是做事，而是很会“进入叙事”。

顺带一提， `deer-flow` 这边我暂时没有查到她自己主动发起 issue 的公开记录。
也就是说，她并不是每个项目都靠 issue 立位；在 DeerFlow 这里，她的可见度主要还是来自 PR 本身。

7. PR / Issue 英文样本：能用，但并不构成她高位形象的强支撑

这部分我一开始其实没有特别在意，但后面把她一些公开 PR / issue 的英文正文抽样看下来后，印象也慢慢稳定了。

如果写得严谨一点，我不会说她“完全不会英文”。
她显然能完成基础的英文标题、PR 描述和简单互动。

但如果有人因为她这些项目经历，就自动脑补成“她还有很强的国际化技术沟通能力”，那当前公开样本也支撑不了这么高的判断。

比较典型的样本包括：

- `apache/fory-site#199` ：**把** `navigator` **写成** `navigatior` **，正文是一整段很长的 run-on sentence**
- `apache/fory-site#198` ： `setting Chinese to default, should be set according to the user's navigator.language`
- `apache/fory-site#193` ： `language quick start Button lost language path`
- `openyurt.io#450` ： `Created the team page ... fetched each person's API information with JSON. git commit`
- `openyurt.io#441` ： `If openyurt website need AOS animations ... I can try to pull request`

这些句子的问题倒不一定严重到“看不懂”，但直译感、模板感、不自然感都比较明显。
而且很多 PR 的 review 互动本身并不多，常常就是发上去后被简单通过，所以它们也不足以支撑“高水平英文协作能力”的印象。

更准确一点的说法应该是：

> 她的英文公开样本是"够用"，但远谈不上这是她形象里一个特别强、特别高位的支撑项。

如果非要继续往下压缩一句，那就是：

> 她真正强的，不是英文，而是选点、行动、进入项目视野，以及把这些东西重新排布成一个更高位的自己。

三、README 对比：她是怎么把自己讲成今天这个版本的

她的 GitHub profile 仓库本身，就是最直观的证据。

这个仓库：

- **总提交大约** `282`
- **其中** `README.md` **就有** `150` **次提交**
- **仅** `2026-03-09` **一天 README 就改了** `13` **次**

如果对比时间线，会看到非常明显的身份升级轨迹。

1. 最早版本：前端学生 / 前端爱好者

`2024-03-17` 创建 README 时，她写的是：

- `I'm interested in front-end development`
- `I'm currently learning React`

这时候她对外最自然的身份是前端方向，这一点非常清楚。

2. 中间版本：前端工程师 + 开源爱好者

后面的 README 逐步出现：

- `A Front-End Development Engineer`
- `Love Open Source`
- `Dream to be Apache Commiter`

这还是一个很典型的前端 / 开源成长线。

3. 2026-03-09 集中重构：AI、Apache、DeerFlow、Infra 一起就位

到 `2026-03-09` 这一轮密集 README 重写，公开叙事突然变得非常完整：

- `AI Full-Stack Developer`
- `Apache Committer`
- `Maintainer of ByteDance/DeerFlow`
- `Currently looking for a full-stack AI agent developer position`
- `Douyin AI4SE RL&Infra`

如果只看这一天的 README 变更轨迹，你几乎能直接看到一个人如何把自己从"前端 / 开源 / 学生成长线"，升级成"AI / Infra / Agent / Apache / DeerFlow / 求职市场高势能候选人"。

这说明什么？

说明她根本不是自然地"把做过的事写上去"，而是在非常清楚地经营一个面向招聘市场和流量市场的个人品牌页。

这也是我为什么一直说，她的问题不是"没有东西"，而是：

> 她太会用有限但真实的东西，包装出一个比真实位置更高的自己。

4. 把 profile README 放回整个个人 GitHub repo 结构里看，会更清楚

如果把 `LofiSu/LofiSu` 这个 profile 仓库先排除掉，只看她账号下其他公开 repo，我看到的就不是"很多长期维护的原创工程"，而是另一种更有结构感的分布。

当前我拉到的非 profile 公共仓库里，大约是：

- `50` 个 fork
- `13` 个原创仓库

也就是说，数量上绝对主体是 fork，而不是原创 repo。

更关键的是，这些 fork 的方向并不是随机分布，而是高度贴着她阶段性的技术叙事和求职叙事走：

- `deer-flow`
- `OpenViking`
- `codex`
- `claude-code`
- `deer-code`
- `MiroFlow`
- `playwright-mcp`
- `spring-ai-alibaba`
- `dify`
- `Flowise`
- `langfuse`
- `ComfyUI`

这条线其实非常清楚：

> AI Agent / MCP / workflow / coding agent / low-code / 可视化 / 前端工具链。

反过来看她自己的原创仓库，最有可见度的大概是这几类：

- `Agent_Kimi_ClaudeCode`： `18` stars， `8` 个 commit
- `myBlog`： `13` stars， `108` 个 commit
- `LLM-NoteBook`： `11` stars， `34` 个 commit
- `InBrowserMcp`： `9` stars， `18` 个 commit
- **其他原创仓库则大多是** `IELTS` 、 `Happy-Agent-Dev-Learning` 、 `mcp-server` 、 `React-Student-Admin` 、 `Huawei-web-Vmall-HTML-CSS-JavaScript` 、 `StickerSmash` 、 `SuKaAI` 、 `lofisu-2026`

这组原创仓库的气质也很稳定：

- 博客
- 笔记
- 教程 / 学习路线
- demo / 小项目
- agent 实战分享
- 个人学习记录

也就是说，个人 GitHub 除了 profile 仓库之外，真正支撑她"原创工程感"的 repo 并不算多。
更明显的结构其实是：

> 她会很快贴近一个上升赛道，fork 一批当下高势能项目，把自己放进这一波技术叙事里；与此同时，再用少量原创内容型 repo 补足"我也在持续产出"的个人品牌结构。

最近的 public events 更能说明这个问题。

如果只看最近 `100` 条公开事件，并且排除 `LofiSu/LofiSu` 这个个人主页仓库，几乎所有公开活跃度都压在 `deer-flow` 这条线上：

- `26` 条 `PushEvent` 落在 `LofiSu/deer-flow`
- `7` 条 `PullRequestEvent` 落在 `bytedance/deer-flow`
- `6` 条 `CreateEvent` 落在 `LofiSu/deer-flow`
- `2` 条 `DeleteEvent` 落在 `LofiSu/deer-flow`
- **其余只剩** `1` **条** `OpenViking` **的 fork 事件，以及 DeerFlow 上的零星 watch / member / comment 事件**

这意味着什么？

意味着她当前最强的公开 coding 活跃度，并不是分散在很多原创 repo 上，而是高度集中在一个高势能的 fork / upstream 叙事里。

如果一个人只看她 profile README，很容易感觉她背后有一个庞大而繁忙的个人开源矩阵；
但把个人 GitHub repo 结构也拆开看以后，会更接近事实的描述其实是：

- **大量 fork 构成她的赛道贴附层**
- **少量原创博客 / 笔记 / demo 构成她的内容产出层**
- **真正近期高频、最能制造"她很活跃"印象的公开 coding 事件，主要集中在 DeerFlow 这条线上**

四、把 README 里的话逐句拆开，会看到什么

她 README 里最值得拆的，不是那些空泛修饰，而是几句带强身份含义的话。

1. `AI Full-Stack Developer`

这句本质上不是客观头衔，而是自我定位。
问题在于，她公开最扎实的作品重心，长期来看仍然明显偏：

- 站点前端
- 文档
- 交互
- Citation / thread / settings
- 页面型和工作流型实现

所以这句更像市场化求职定位，而不是一个外部世界已经替她盖章的客观身份。

2. `Apache Committer`

这句成立。
但它最强的公开证据，主要支撑的是 Apache 官网/站点建设，而不是主仓库核心实现。

所以如果有人顺着这句话自动脑补成"她深度参与的是 Apache Fory 核心引擎"，那就已经超过公开证据本身。

3. `Maintainer of ByteDance/DeerFlow`

这句里，"在 DeerFlow 有真实贡献"是真的。
但"Maintainer"这个词，按当前能公开核实到的页面，证据仍然不够。尤其是 DeerFlow README 里公开列出来的 `Key Contributors / core authors` 并没有她。

所以更稳妥的说法应该是：她是活跃贡献者，而不是我能直接替她盖章的公开 maintainer。

4. `Tencent Rhino Bird ... Tencent Omi`

`Omi contributor` 这半句成立。
但如果把整句话读成"她在 Omi 这边有分量很重的长期项目角色"，那就是放大。公开代码侧支撑的，只是一条体量不大的模板页面 PR 线。

至于"犀牛鸟课题实战奖"本身，我目前只能说她有自述和截图，尚未找到独立官方公示来完成外部核验。

5. `Alibaba Open Source Contributor — Higress · OpenYurt · Spring AI Alibaba`

这句是她 README 里最典型的一种写法。

因为：

- `Spring AI Alibaba` **：公开证据不为空**
- `Higress` **：主仓库公开证据当前是** `0`
- `OpenYurt` **：主仓库公开证据当前也是** `0`

如果她写的是 `Spring AI Alibaba Contributor` ，我不会说这句完全站不住。
但把三者并列打包成一句统一的 `Alibaba Open Source Contributor` ，就明显比当前主仓库公开证据更满。

五、从事实切入，我会如何描述这个人

如果我刻意不带情绪，只根据这些公开记录来描述她，我会这么写：

她是一个有真实公开产出的人，而且不是零散产出。她在 Apache Fory Site、DeerFlow、Omi、Spring AI Alibaba 这些地方都留下了一定程度的公开痕迹。她最扎实的工作重心长期集中在站点前端、文档、交互、线程状态、Citation、页面型和可视化型实现。她也会主动通过 issue 进入项目视野，先提出需求、方案或切入口，再把 PR 和履历接到这条线上。她的个人 GitHub repo 结构也很说明问题：大量 fork 负责贴近热点和赛道，少量原创博客 / 笔记 / demo 负责补足持续产出感。她很会整理这些材料，也很懂怎么把它们重新排列成一套更高位的身份叙事。

如果一定要浓缩成一句话，那就是：

> 她不是没东西。她是太会用有限但真实的东西，包装出一个比真实位置更高的自己。

六、为什么我最后还是对这种人反感

如果事情只停留在"她很会包装"，其实我未必会这么反感。

因为这个时代里，会包装的人太多了。尤其在 AI 圈，这几乎已经成了半公开的默认技能。
真正让我反感的，不是她会讲故事，而是她讲故事的方式，和她建立 connection 的方式，看起来越来越像同一套机制：

- **快速占位**
- **快速抬升**
- **快速连接**
- **快速筛选**
- **快速丢弃**

而且这种"快速占位"，不只体现在私下社交里，也体现在她对开源项目的进入方式里：先找一个前端/官网型切口，用 issue 试探、用 PR 落地、用 README 抬位，最后把这些动作拼成一个更强的外部角色。

如果没有我和她那次私下交集，我对她的判断可能停留在：

> 这个人很会做品牌、很会做 connection，也很懂怎么在 AI 叙事里给自己抬位。

但加上那段私人经历之后，我对她的判断就变成了：

> 这个人的社交方式带有很强的工具性。她不是在稳定建立关系，而是在快速扫描、快速接近、快速筛选，然后快速丢弃。

这也是为什么我对她的最终判断不是"骗子"，而是另一种我同样不喜欢的人：

> 她是那种很懂当代技术圈游戏规则、也很会借这些规则往上走的人。她有真实产出，但也很会把这些产出包装成更强的身份；她会建立 connection，但这种 connection 给人的感觉更像资源调度操作，而不是关系本身。这样的人可以很会赢，但未必很值得尊重。

七、如果把视角再拉高一点：我会怎么评价这类人

如果我不只是在评价她，而是在评价"她所代表的这一类人"，我会把这类人定义为：

> 高执行力、强叙事、强机会感，但边界感偏弱的上升型人格。

如果再把这种人的印象拆成三个层级，我会觉得大概是这样：

1. 第一眼印象

第一眼看过去，往往会觉得这种人：

- **很会做事**
- **很会表达**
- **很会抓机会**
- **履历很满**
- **好像到处都在参与，而且参与得都不浅**

也就是说，第一眼最容易形成的感觉通常不是"假"，而是：

> 这个人很会、很活跃、很上进，而且上升势头很强。

这也是为什么这类人很容易在短时间内获得关注。
因为她们给外界提供的是一种高密度、高完成度、高势能的整体印象。

2. 细挖一下后的印象

当你开始看具体 PR、issue、repo、时间线、贡献边界之后，印象就会发生第一次变化。

这时你通常会发现：

- **她不是空壳，确实做过一些东西**
- **但很多"强身份"背后的真实重心，比表面印象更窄**
- **很多经历并不是假的，而是被排布得特别会说话**
- **她最强的能力未必是单点技术深度，而是选点、占位、进入叙事、放大观感**

也就是说，细挖一下后的感觉不再是"哇，这个人好强"，而更像是：

> 她不是没有东西，她是特别会把现有的东西讲成一个更强的自己。

这一步其实最容易让人进入一种复杂感受：
你开始不舒服，但还不至于一句话把她打成假的，因为她确实有真实材料。

3. 彻底深挖后的印象

等你把公开贡献、issue 行为、个人 GitHub repo 结构、README 迭代、英文样本、以及她和人建立 connection 的方式全部放在一起看，印象会再变一次。

这时她给人的感觉通常不再只是"很会包装"，而会更接近：

- **她很懂系统奖励什么**
- **她很懂如何在边界内抬升观感**
- **她很懂怎样把项目、头衔、社交都变成上升材料**
- **她不是在稳定建立位置，而是在持续经营一个可被市场高估的位置**

到这一步，真正让人不适的东西就不只是包装本身，而是：

> 她整个人像一个被高度经营过的上升装置。

你会开始感觉，她不是单纯在"表达自己"，而是在"设计别人如何理解她"；
她不是单纯在"建立关系"，而是在"配置资源接口"；
她不是单纯在"积累经历"，而是在"制造一个高于事实边界的自己"。

这也是为什么这类人越深挖，往往越容易让人反感。
不是因为你发现了某条惊天假话，而是因为你越来越清楚地看见：

> 她的很多动作，单独看都可以解释；但全部拼起来，就是一套非常成熟、非常工具化、也非常不让我尊重的上升机制。

她们通常不是空壳，也不是完全不会做事。
恰恰相反，她们往往很会读环境、很会选赛道、很会找切口，也很会在一个项目里迅速找到"最容易被看见、最适合自己发挥、又最容易转化成身份叙事"的位置。

所以如果只看能力面，我会承认这类人有几种真实且不弱的能力：

- **会判断市场奖励什么**
- **会选择高势能叙事**
- **会快速进入项目视野**
- **会把碎片化经历整理成连续故事**
- **会把"事实正确"与"印象抬升"之间的空间利用到极致**

但问题也恰恰出在最后一点。

她们最让人不舒服的，不一定是"造假"，而是：

> 每一条单独拿出来，可能都不完全错；但整体拼起来，就是在持续诱导别人高估她的位置。

这会带来一种很强的不适感，因为它不是赤裸裸的欺骗，而是一种边界被精确操控后的失真。
你很难一句话把她打成"假的"，但你又能明显感觉到，她呈现出来的自己，已经高于公开事实真正能稳稳支撑的位置。

如果这种模式再进一步进入社交关系里，就会更让人本能反感。
因为这时候被工具化的，不只是头衔、项目和履历，还有人本身。

于是这类人会呈现出一个很典型的双重性：

- **从功利结果看，她们往往是有效的**
- **从信任和尊重看，她们往往是可疑的**

我会承认她们"很会赢"，但我不会轻易把她们归到"真正值得尊重的强者"里。
因为我对强者的标准从来不只是会不会往上走，而是：

- **能力和位置是否匹配**
- **表达和事实是否对齐**
- **关系和人是否被当成纯资源**

所以如果非要再压成一句最短的判断，那就是：

> 这类人通常很会经营形象，也很会利用系统，但未必配得上她们试图让别人相信的那个自己。

而我对这类人的本能不适，本质上也不是"讨厌会包装的人"，而是：

> 我厌恶一种把能力、关系、身份都变成可操作展示品的上升方式。

八、结论

如果只问一句：LofiSu 有没有完全胡说八道？

没有。她有不少真实材料。

但如果问得更准确一点：

她是不是很会把真实但边界明确的材料，重新排列、重新包装，最后产出一个比事实位置更高的自己？

我的答案是：是，而且这几乎就是她最核心的能力。

说得再直白一点：

她最厉害的地方，不是她写了多少假话，而是她太会用有限但真实的话，制造出一个明显高于事实边界的整体印象。

参考

- 当前 README：https://github.com/LofiSu/LofiSu/blob/main/README.md
- GitHub 主页：https://github.com/LofiSu
- GitHub 仓库列表：https://github.com/LofiSu?tab=repositories
- 2024-03-17 初始 README 提交：https://github.com/LofiSu/LofiSu/commit/95eb84708fb586d2a6de8726fa217b7f8171984c
- 2026-03-09 README 重构提交：https://github.com/LofiSu/LofiSu/commit/47ebda5cf43b23b1b37cf17dcce7f131411b49ed
- 2026-03-09 README 重构提交：https://github.com/LofiSu/LofiSu/commit/660710f6042bffa27f6a9ac0406e7f5bb76ac013
- `Agent_Kimi_ClaudeCode` ：https://github.com/LofiSu/Agent_Kimi_ClaudeCode
- `myBlog` ：https://github.com/LofiSu/myBlog
- `LLM-NoteBook` ：https://github.com/LofiSu/LLM-NoteBook

- `InBrowserMcp`：https://github.com/LofiSu/InBrowserMcp
- ASF committer 名单：https://home.apache.org/committer-index.html#lofisu
- ASF 项目名单：https://home.apache.org/committers-by-project.html#fory
- Apache Fory Site commits by author：https://github.com/apache/fory-site/commits/main/?author=LofiSu
- Apache Fory Site PR 搜索：https://github.com/apache/fory-site/pulls?q=is%3Apr+author%3ALofiSu
- Apache Fory Site PR `#141`：https://github.com/apache/fory-site/pull/141
- Apache Fory Site PR `#147`：https://github.com/apache/fory-site/pull/147
- Apache Fory Site PR `#152`：https://github.com/apache/fory-site/pull/152
- Apache Fory Site PR `#177`：https://github.com/apache/fory-site/pull/177
- Apache Fory Site PR `#181`：https://github.com/apache/fory-site/pull/181
- Apache Fory Site PR `#193`：https://github.com/apache/fory-site/pull/193
- Apache Fory Site PR `#198`：https://github.com/apache/fory-site/pull/198
- Apache Fory Site PR `#202`：https://github.com/apache/fory-site/pull/202
- Apache Fory Site PR `#213`：https://github.com/apache/fory-site/pull/213
- Apache Fory Site PR `#216`：https://github.com/apache/fory-site/pull/216
- Apache Fory Site PR `#264`：https://github.com/apache/fory-site/pull/264
- Apache Fory main commits by author：https://github.com/apache/fory/commits/main/?author=LofiSu
- Apache Fory PR `#1867`：https://github.com/apache/fory/pull/1867
- DeerFlow repo：https://github.com/bytedance/deer-flow
- DeerFlow PR `#1063`：https://github.com/bytedance/deer-flow/pull/1063
- DeerFlow PR `#1064`：https://github.com/bytedance/deer-flow/pull/1064
- DeerFlow PR `#1077`：https://github.com/bytedance/deer-flow/pull/1077
- DeerFlow PR `#1081`：https://github.com/bytedance/deer-flow/pull/1081
- DeerFlow PR `#1088`：https://github.com/bytedance/deer-flow/pull/1088
- DeerFlow README `Key Contributors`：https://github.com/bytedance/deer-flow#key-contributors
- Tencent Omi PR `#913`：https://github.com/Tencent/omi/pull/913
- Tencent Omi PR `#914`：https://github.com/Tencent/omi/pull/914
- Spring AI Alibaba PR `#189`：https://github.com/alibaba/spring-ai-alibaba/pull/189
- Spring AI Alibaba PR `#208`：https://github.com/alibaba/spring-ai-alibaba/pull/208
- Apache Fory Site issue `#139`：https://github.com/apache/fory-site/issues/139
- Apache Fory Site issue `#149`：https://github.com/apache/fory-site/issues/149
- Apache Fory Site issue `#212`：https://github.com/apache/fory-site/issues/212
- Apache Fory issue `#1793`：https://github.com/apache/fory/issues/1793
- Spring AI Alibaba issue `#162`：https://github.com/alibaba/spring-ai-alibaba/issues/162
- Spring AI Alibaba issue `#170`：https://github.com/alibaba/spring-ai-alibaba/issues/170
- Spring AI Alibaba issue `#619`：https://github.com/alibaba/spring-ai-alibaba/issues/619
- Higress 主仓库：https://github.com/alibaba/higress
- Higress 官网 issue `#287`：https://github.com/higress-group/higress-group.github.io/issues/287
- Higress 官网 issue `#294`：https://github.com/higress-group/higress-group.github.io/issues/294
- Higress 官网 issue `#306`：https://github.com/higress-group/higress-group.github.io/issues/306
- OpenYurt 主仓库：https://github.com/openyurtio/openyurt
- OpenYurt 官网 issue `#441`：https://github.com/openyurtio/openyurt.io/issues/441
- OpenYurt 官网 issue `#443`：https://github.com/openyurtio/openyurt.io/issues/443
- OpenYurt 官网 issue `#446`：https://github.com/openyurtio/openyurt.io/issues/446

作者提示: 个人观点，仅供参考

Yonagi04

赞 分享 推荐 写留言